AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515**, hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application
for STLC Automation

represents work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** and **November, 2026**. All external sources and prior work used in preparing this report are cited.
With my signature, I certify that:

- I have not manipulated any data or results reported in this document.
- I have clearly indicated and referenced the contributions of others.
- I have acknowledged collaborative research and discussions relevant to this work.
- I understand that false claims or unacknowledged use of intellectual property may result in disciplinary action.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge.

Internal Guide: *Kapil Madaan* Internal Guide Signature

Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor: *Punit Rathore* Faculty Mentor Signature

Department: *Cyber Physical Systems*

© Monika Tyagi
November, 2026
All rights reserved

Acknowledgements

The author acknowledges the guidance of Mr. Kapil Madaan, internal project guide, and Prof. Punit Rathore, faculty mentor, in defining and pursuing this project. The author also thanks the faculty and staff of the Indian Institute of Science for supporting the M.Tech. (Online) programme and the academic review process.

Abstract

The Software Testing Life Cycle (STLC) spans requirements analysis, test planning, test design, test execution, defect reporting, and release decisions. In many teams, these activities are distributed across tools and depend on manual hand-offs. This project investigates an AI-assisted approach that connects quality-engineering tasks through a central orchestrator and a set of focused agents. The objective is to help practitioners transform requirements, user stories, code changes, and test-run evidence into actionable quality guidance while retaining human responsibility for review and release decisions.

The implemented prototype is a web application composed of a React and TypeScript frontend, a FastAPI backend, a PostgreSQL database, and a separate agent service built with Google Agent Development Kit (ADK). The backend authenticates users, records assistance requests, and calls the agent service. The root agent can route work to nine STLC specialists covering test-case generation, test-data generation, script-automation suggestions, execution analysis, self-healing recommendations, change-impact analysis, defect reporting, test-summary reporting, and release-readiness assessment. The application also supports Jira Cloud story import, GitHub diff retrieval, optional Microsoft Entra single sign-on, and multiple model-provider configurations.

The report describes the motivation, related testing and agent literature, architecture, implementation, security considerations, and a reproducible evaluation plan. It distinguishes the capabilities present in the repository from the broader proposal: test-execution analysis does not itself run an external test suite, script-healing output is advisory, and no completed comparative benchmark or measured improvement was supplied for this report. Evaluation is therefore framed as future work using human-reviewed artifacts, traceability, effort, execution-time, and

defect-analysis measures. The prototype demonstrates an integrated application workflow; its effectiveness and operational suitability require further controlled and organizational validation.

Contents

Acknowledgements i

Abstract ii

Contents iv

List of Figures vii

List of Tables viii

List of Algorithms ix

Acronyms, Abbreviations and Symbols xi

1 Introduction 1

1.1 Background 1

1.2 Problem Statement and Motivation 2

1.3 Research Questions 2

1.4 Objectives 2

1.5 Scope and Contributions 3

1.6 Report Organization 4

2 Literature Review 5

2.1 Software Testing Life Cycle 5

2.2 LLMs in Software Quality Engineering 5

2.3 Agent-Orchestrated Workflows . . . 6
2.4 Grounding, Traceability, and Human Review . . . 6
2.5 Evaluation of AI-Assisted Testing . . . 7
2.6 Research Gap and Project Position . . . 7

3 Methodology and Implementation **8**
3.1 Development Method . . . 8
3.2 Requirements and Actors . . . 8
3.3 System Architecture . . . 9
3.3.1 Frontend . . . 10
3.3.2 Backend and Persistence . . . 10
3.3.3 Agent Orchestration . . . 11
3.3.4 External Integrations and Model Providers . . . 14
3.4 Security and Data Handling . . . 14
3.5 Deployment and Operations . . . 14
3.6 Quality Attributes . . . 15

4 Results and Discussions **16**
4.1 Prototype Implementation Status . . . 16
4.2 Existing Automated Test Assets . . . 16
4.3 Demonstration Scenarios . . . 16
4.3.1 Interface Walkthrough and Illustrative Outputs . . . 18
4.4 Evaluation Plan . . . 23
4.4.1 Dataset and Protocol . . . 23
4.4.2 Measures . . . 24
4.4.3 Analysis and Validity . . . 24
4.5 Results and Discussion . . . 25
4.6 Limitations . . . 25

5 Conclusions **27**

5.1 Future Work 27

Appendix **29**

5.A Representative Request and Response Flow 29

5.A Reproducibility Checklist for a Future Evaluation 29

5.A Project Materials 30

References **31**

List of Figures

3.1 Logical application architecture. Optional external integrations and model providers are enabled by configuration. 9

3.2 Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository. 10

3.3 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 12

3.4 Illustrative request sequence through the application services for the deployment configuration shown. 13

4.1 Workspace dashboard used to start a quality-assistance request. 19

4.2 Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials. 20

4.3 Example request input containing quality context and a user story before agent processing. 20

4.4 Example generated response with prioritized test-case recommendations. 21

4.5 Example suggested test data presented as part of the generated response. 21

4.6 Example automation notes and test-script recommendations. 22

4.7 Example release-risk and readiness analysis based on the submitted context. . . 23

List of Tables

3.1 Selected functional requirements and corresponding implemented behavior. . . . 9
3.2 Registered specialist agents and the scope described by their instructions. 11

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 17
4.2 Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed. 24

List of Algorithms

1 Assistance request processing 13

Acronyms, Abbreviations and Symbols

Key terms used throughout this report are listed below.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Software quality is produced through a sequence of connected activities rather than by test execution alone. A typical Software Testing Life Cycle (STLC) interprets requirements, plans test scope, designs cases and data, executes checks, reports defects, summarizes evidence, and informs release decisions. Testing standards and practitioner guidance emphasize that these activities should be planned, traceable, and adapted to risk [1, 2].

In practice, quality information is often distributed across issue trackers, source-control systems, test frameworks, spreadsheets, and team discussions. Requirements may be restated manually as test cases; code changes may not be consistently mapped to regression scope; and results from a test run may not be converted into a concise, risk-aware release summary. Repetition and inconsistent interpretation can consume engineering time and weaken traceability.

Large language models (LLMs) can assist with language-intensive transformations such as summarizing a requirement, proposing boundary cases, or organizing failure logs. They do not, by themselves, guarantee factual correctness, completeness, or safe execution. A useful quality assistant must therefore combine contextual input, focused task instructions, explicit output expectations, access controls, and human review.

1.2 Problem Statement and Motivation

The motivating problem is the lack of a connected, intelligent workflow that helps a quality team move from change context to test and release evidence without repeating the same interpretation at every stage. Existing automation can be effective at individual tasks, while upstream requirements analysis and downstream defect triage or readiness assessment may remain dependent on manual work. The resulting gaps can lead to omitted scenarios, duplicated effort, and late discovery of risk.

The project proposes an orchestrated assistant for STLC-related work. Users submit requirements and user stories, and may provide a source-code diff or execution evidence. A root agent selects relevant specialists and returns a synthesized response. The goal is decision support: the system drafts and organizes quality artifacts; a qualified engineer remains responsible for confirming test correctness, executing tests in the appropriate environment, and deciding whether a release is acceptable.

1.3 Research Questions

The project is guided by three questions adapted from the proposal and mid-term presentation [3, 4]:

1. **RQ1:** How can a multi-agent AI system coordinate multiple STLC activities through a single quality-assistance interface?

2. **RQ2:** What validation, provenance, and human-review controls are needed to make AI-generated testing artifacts more reliable and traceable?

3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The objectives are to:

- design and implement a full-stack application for submitting software-quality context and reviewing AI-assisted results;
- implement a central agent with specialist capabilities for common STLC tasks;
- persist user-scoped requests and provide authenticated access to the application;
- connect selected engineering context sources, including Jira issues and GitHub code changes;
- support configurable model providers and deployment settings; and
- define a reproducible, human-reviewed evaluation plan without claiming unmeasured benefits.

1.5 Scope and Contributions

This report covers the application and agent implementation present in the project repository as of November 2026. Its contribution is a working, modular prototype that connects a browser-based interface, a persistent API, and a specialized agent service. The implemented interface accepts requirements and user stories, provides a place for code diffs and execution logs, supports follow-up turns, and renders or exports generated responses. Optional integrations provide Jira Cloud issue context and GitHub diffs. The agent service registers nine STLC specialists and configures a root orchestrator.

The project does not claim that the prototype autonomously executes a user's test suite, changes scripts in a repository, proves that generated cases are complete, or makes an authoritative release decision. Execution, script-healing, and readiness responses are analytical suggestions based on supplied context. Retrieval-augmented generation over a persistent test repository, automated CI/CD execution, formal output validation, and controlled comparative evaluation remain future work unless otherwise configured outside the repository.

1.6 Report Organization

Chapter 2 reviews the testing lifecycle, LLM-assisted testing, agent orchestration, and validation concerns. Chapter 3 describes the development method, system architecture, implementation, and security model. Chapter 4 records the implementation evidence available and defines the evaluation needed to answer the research questions. Chapter 5 summarizes the work and identifies limitations and next steps.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Software testing is a planned set of activities that evaluates software products and provides evidence about quality. The ISO/IEC/IEEE 29119 series describes concepts, processes, and documentation for software testing, while the ISTQB Foundation syllabus presents a practitioner-oriented view of test planning, analysis, design, implementation, execution, and completion [1, 2]. Across these approaches, a recurring principle is traceability: test conditions and results should relate to requirements and risks, and conclusions should be based on recorded evidence.

The STLC is not always a strictly linear sequence. Test results may reveal unclear requirements; a defect may lead to new regression cases; and a code change may alter the appropriate scope of testing. A system that assists with several phases should therefore preserve context and make transitions understandable, rather than treating each generated artifact as an isolated answer.

2.2 LLMs in Software Quality Engineering

LLMs offer flexible text understanding and generation, which makes them suitable for drafting test scenarios from natural-language requirements, proposing data partitions, classifying failure descriptions, and summarizing test runs. These are useful capabilities for reducing repetitive composition work. The same flexibility creates risks: a model can omit constraints, invent

details, generate invalid steps, or state conclusions more confidently than the evidence supports.

Work on machine-learning-enabled software engineering emphasizes the need for human-centered workflow design, clear ownership, data quality, testing, and continuous monitoring [5]. In a testing assistant, model output should be treated as a candidate artifact. A practitioner should be able to check assumptions, revise generated cases, and reject unsupported recommendations before they affect a test plan or release decision.

2.3 Agent-Orchestrated Workflows

An agent architecture can assign distinct responsibilities to specialized components and use a coordinating component to select or combine them. The ReAct approach studies interleaving reasoning with actions, while surveys of LLM agents discuss planning, tool use, memory, and coordination as core design dimensions [6, 7]. Specialization can improve task focus and make prompts easier to maintain, but it does not automatically produce correct outputs. Additional model calls also introduce latency, cost, and coordination failure modes.

The present project applies this pattern through a root agent and a registry of STLC specialists. Routing is based on the submitted work context and the request's intent. The architecture is modular, but the current implementation primarily coordinates through agent instructions and generated text. It should not be confused with a deterministic workflow engine or a formally verified pipeline: explicit schemas, validation gates, and a user approval interface for each stage are areas for further work.

2.4 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) combines a language model with retrieval from an external knowledge source. The original RAG work shows how retrieved documents can supply context for knowledge-intensive generation [8]. For software testing, retrieval could ground generation in approved requirements, test policies, API specifications, or prior defects. The project proposal and mid-term presentation identify this as a future direction; the repository inspected for this report does not implement a persistent retrieval index for test assets. The current prototype relies on the context supplied in each request and any available agent session

context.

Grounding is only one part of reliability. An evaluation should distinguish source traceability from factual correctness, and should record the requirement or code change that motivated each suggested test. Human review can identify omissions and unsupported assertions. For higher-impact decisions, the workflow should preserve the evidence behind the result, expose uncertainty, and require an explicit human decision.

2.5 Evaluation of AI-Assisted Testing

A credible evaluation needs a defined baseline, representative inputs, consistent model and prompt settings, and measurable outcomes. Proposed measures include requirement-to-test traceability, scenario coverage, human-rated artifact accuracy, time spent on comparable tasks, defect-classification quality, and end-to-end response latency. These measures answer different questions; for example, more generated cases do not necessarily mean better coverage, and faster drafting does not necessarily mean less total review effort.

The NIST AI Risk Management Framework recommends managing AI risks through governance, mapping, measurement, and management activities [9]. For this project, that perspective supports recording model and configuration versions, defining review criteria, separating model suggestions from verified results, and assessing privacy and operational risks alongside productivity metrics.

2.6 Research Gap and Project Position

The problem framing in the project proposal and mid-term presentation is that QA automation frequently emphasizes isolated activities, whereas teams also need coordination between requirements, test design, changes, failures, and release evidence. This project addresses the integration gap by building one application with a coordinating agent and task-specialized capabilities. It is an engineering prototype, not a claim that agent orchestration is itself novel across the research field. Its research value depends on demonstrating useful coordination, dependable output, and measurable improvement against an explicit baseline.

Chapter 3

Methodology and Implementation

3.1 Development Method

The project follows an iterative software-engineering approach: define the quality-workflow problem, design a service-oriented prototype, implement the user-facing and agent paths, and identify evidence required for evaluation. This approach is appropriate for a project whose artifact is an end-to-end application. The proposal describes a phased plan using public and open-source projects before any controlled use of anonymized organizational artifacts, and the mid-term presentation records the earlier prototype and evaluation status [3, 4]. This report describes the current implementation and retains comparative evaluation as planned work.

The implementation was examined across the frontend, backend, agent service, tests, and deployment documentation. Claims about current functionality in this chapter are limited to those code paths and user-facing flows. The mid-term presentation is used as an earlier statement of research objectives, while the project proposal provides the initial scope and evaluation plan.

3.2 Requirements and Actors

The principal actor is a quality engineer, developer, or release stakeholder who needs structured testing guidance. The user authenticates, supplies requirements or stories, optionally adds a code diff or test-run log, and reviews the generated response. A follow-up can reuse a session

identifier. An administrator configures the model provider and optional integrations through environment settings.

Requirement	Prototype behavior
Context submission	Accepts requirements, user stories, code diffs, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch-diff retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 System Architecture

The application separates the browser interface, application API, database, and agent runtime. The frontend is implemented in React and TypeScript and communicates with the FastAPI backend. The backend validates user access, stores assistance requests, constructs the message sent to the agent, and records the agent response. The separate agent service uses Google ADK and a model abstraction to run the root agent and its specialists. PostgreSQL stores user, request, and integration records.

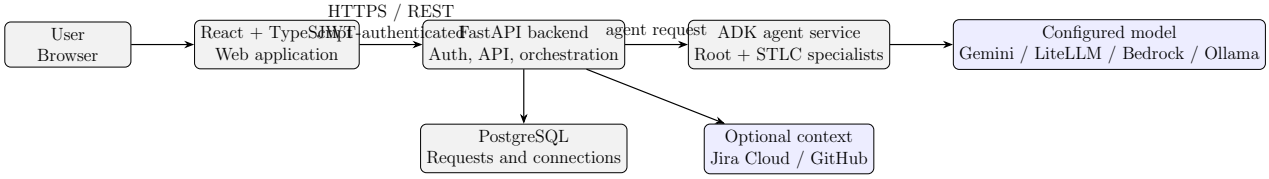


Figure 3.1: Logical application architecture. Optional external integrations and model providers are enabled by configuration.

The logical architecture describes the software components implemented in the repository. The following AWS diagram is a proposed deployment target rather than evidence of a de-

ployed production environment; the repository currently provides deployment guidance, not infrastructure provisioning.

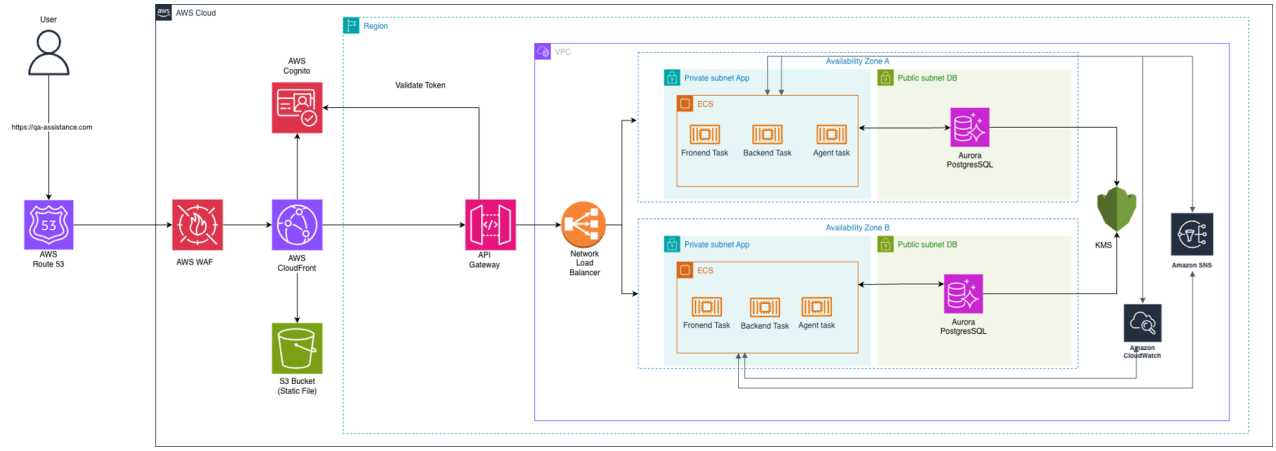


Figure 3.2: Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository.

3.3.1 Frontend

The frontend provides authentication, a dashboard for quality-assistance requests, settings, and user guidance. The dashboard accepts free-form requirements and user stories, supports code-change input, and includes sample requests for smoke planning, end-to-end examples, change impact, test execution and reporting, automation, and follow-up turns. When connected, Jira issues can populate the story context and GitHub can supply a pull-request or compare diff. Responses are rendered in the browser and can be downloaded as Markdown or HTML.

3.3.2 Backend and Persistence

The backend is a FastAPI service backed by asynchronous SQLAlchemy and PostgreSQL [10]. The `/api/assist` route requires an authenticated user. It stores a request with its owner, source text, session identifier, status, timestamps, and eventual response. It constructs a structured agent message and calls the agent service. Success and failure states are recorded, and errors are returned to the frontend using the API's error-handling path.

The persistence model separates users, assistance requests, Jira connections, and GitHub

repository connections. Stored requests provide an application record of prior submissions. This is distinct from the conversational state maintained by the agent runtime: the current ADK runner uses in-memory session storage, so agent context should not be treated as a durable conversation archive across service restarts.

3.3.3 Agent Orchestration

The root agent is named `quality_assistance_agent`. It is built with Google ADK, a configured model, logging callbacks, and available tools [11]. For supported providers, the root agent can transfer work to the registered sub-agents; the local Ollama configuration uses a consolidated instruction path without those sub-agents. The backend constructs a prompt from requirements, stories, and code changes and requests prioritized test cases, suggested data, automation notes, and release risks.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code diffs to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.2: Registered specialist agents and the scope described by their instructions.

The router's instructions map requirement and story requests to test-case and test-data agents, code diffs to change-impact analysis, execution logs to execution, defect, and summary

agents, and release questions to the readiness agent. The root agent can also use a requirement-summarization tool. This is prompt- and agent-driven routing, not a separate deterministic workflow engine.

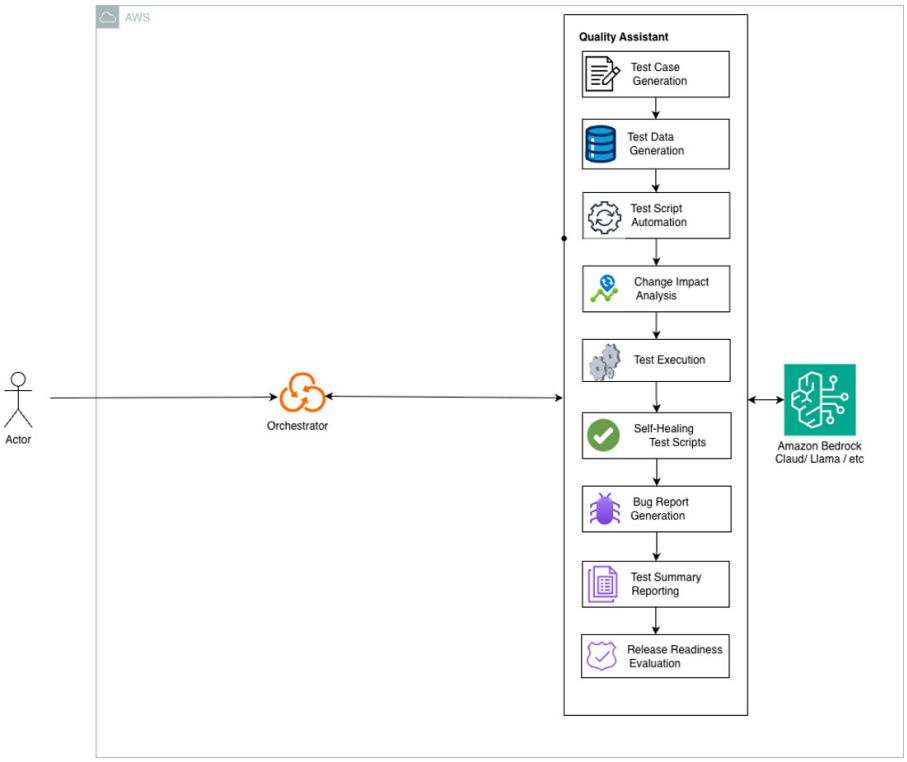


Figure 3.3: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

Algorithm 1 Assistance request processing

Require: Authenticated user u, requirements r, optional stories s, diff d, session q

Validate the request and associate it with u
Persist request inputs with status `processing`
Construct a message from r, s, and d
Resolve or create an agent session for u
Invoke the root agent with the message and session
Route to one or more specialists according to the request context
Collect the generated response from the agent events
if agent call succeeds **then**
    Persist session identifier, response, and status `completed`
else
    Persist failure status and return an explicit service error
end if
Return the request identifier, session, response, and status

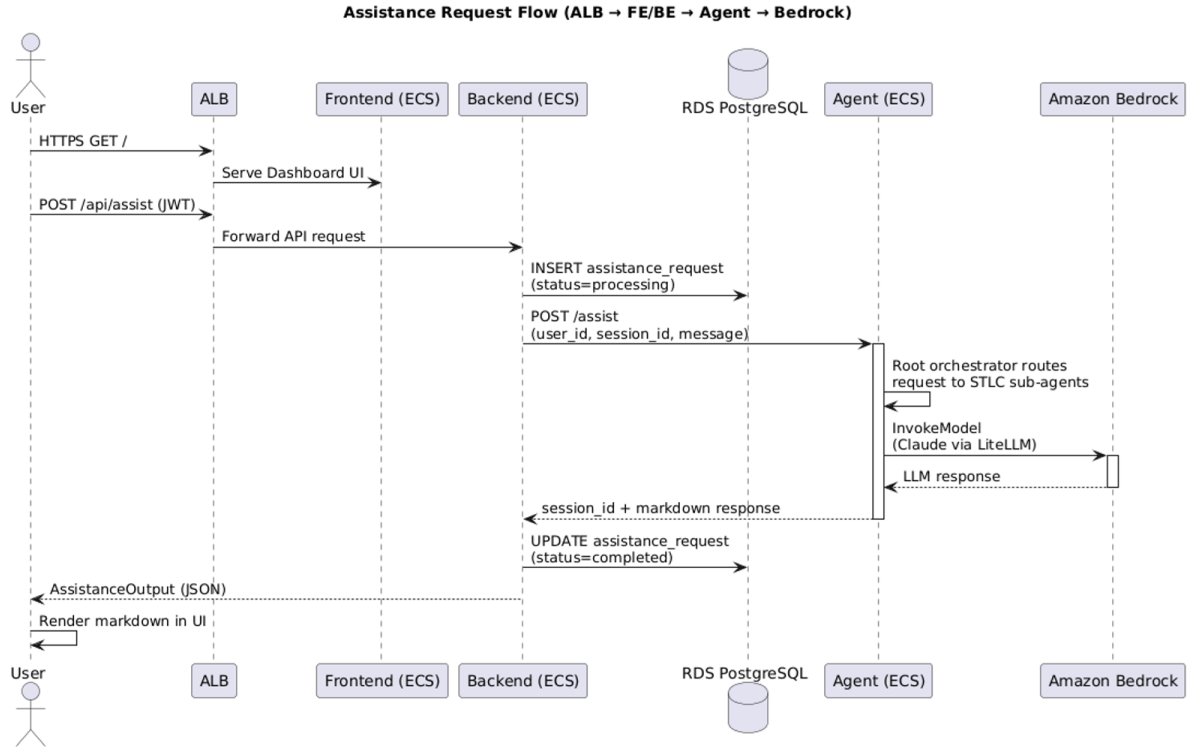


Figure 3.4: Illustrative request sequence through the application services for the deployment configuration shown.

3.3.4 External Integrations and Model Providers

Jira Cloud connection uses OAuth 2.0 authorization, state records, and Jira APIs to retrieve a user's assigned issues and issue details. GitHub connection settings store a repository and an encrypted personal access token; the backend can fetch pull-request diffs or compare two references. Microsoft Entra single sign-on is an optional authentication path. The agent configuration accepts Gemini, LiteLLM, Amazon Bedrock, and Ollama backends, subject to provider credentials and model availability. These are configuration options, not a claim that all providers have been benchmarked under the same conditions.

3.4 Security and Data Handling

The backend scopes assistance records to an authenticated user and issues JWT access tokens after successful registration or login. Password handling is delegated to the project's password manager. Jira and GitHub credentials are stored in encrypted form; Jira OAuth state is represented by a digest and expires. Production settings validate that sensitive secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials are supplied. The ECS guide describes a deployment pattern with private application and database services, IAM roles, and managed secrets.

The submitted requirements, stories, code diffs, and test results may contain proprietary information. They are persisted with the request and sent to the configured model provider for inference. Deployers must therefore select providers and retention settings consistent with organizational policy, limit access to stored artifacts, rotate credentials appropriately, and avoid submitting secrets. The prototype's security controls reduce exposure but do not replace a privacy impact assessment, threat model, independent security review, or organization-specific data-governance approval.

3.5 Deployment and Operations

The repository supports a local full-stack Docker Compose configuration for PostgreSQL, frontend, backend, and agent services. Development scripts are also provided for a local frontend

and Python services with a local or containerized database. Deployment documentation describes an AWS ECS Fargate pilot using separate containers, RDS PostgreSQL, an Application Load Balancer, and a configured model provider. That document is an operational plan and checklist rather than an infrastructure-as-code deployment; this report does not claim that an AWS production environment has been deployed.

3.6 Quality Attributes

- **Modularity:** frontend, API, persistence, and agent service are separated; specialists are registered independently.
- **Configurability:** model providers, endpoints, integrations, and production settings are selected through environment configuration.
- **Traceability:** the backend persists request inputs and responses by user; detailed requirement-to-case evidence and durable agent memory are not yet formalized.
- **Availability:** health and readiness endpoints and container deployment guidance support operations; production resilience depends on the selected deployment topology.
- **Reliability:** explicit backend error paths and settings validation exist, while generated quality artifacts still require human review and empirical evaluation.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

The repository contains a functional full-stack prototype rather than only the architecture proposal. The implemented product path includes user registration and login, an authenticated assistance API, persistent request records, an agent service, a dashboard for submitting quality context, and settings for external connections. Nine specialist agents are registered and selected through the root agent's routing instructions. GitHub diff retrieval, Jira Cloud OAuth and issue retrieval, and optional Microsoft Entra authentication are present in the source.

4.2 Existing Automated Test Assets

The repository includes focused test modules for agent Bedrock credential settings, backend production configuration, Jira Cloud service behavior, and Microsoft Entra SSO behavior. The test assets provide evidence that provider configuration and selected integration paths are being tested. Their existence is not equivalent to a passing run or to end-to-end validation of every UI, model, database, or agent workflow. A final submission should record the exact test command, environment, commit, test count, and pass/fail outcome from a reproducible run.

4.3 Demonstration Scenarios

The dashboard includes sample inputs that can be used to demonstrate several agent routes:

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support diff retrieval.
Authentication and deployment controls	Implemented in part	Password/JWT authentication, optional Entra SSO, encrypted integration credentials, and production settings validation are present.
RAG over organizational test assets	Planned	No persistent requirements/test/defect retrieval index is evidenced in the inspected implementation.
Automated CI test execution and script mutation	Planned / out of scope of current behavior	Execution and healing agents analyze or recommend; they do not execute external suites or commit changes.
Comparative empirical evaluation	Pending	No measured baseline, benchmark results, or controlled organizational pilot results were supplied.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

1. a password-reset requirement for test planning and requirement summarization;
2. an e-commerce checkout scenario with requirements, user stories, and a code diff;
3. a diff-heavy authentication change for regression impact analysis;
4. a test-run log containing pass, fail, and skipped cases for execution analysis and reporting;
5. a Playwright selector change for automation and self-healing recommendations; and
6. a follow-up request that reuses the prior session identifier.

These examples verify that the interface and request model support representative kinds of context. They are not a benchmark dataset and do not establish measured improvements in coverage, effort, or defect detection.

4.3.1 Interface Walkthrough and Illustrative Outputs

The screenshots below follow the user workflow: the workspace and integration settings are shown first, followed by the submitted requirements and user story, and then the response panels. They document the prototype interface and example generated content; they are not results from a controlled evaluation, and generated recommendations require human review.

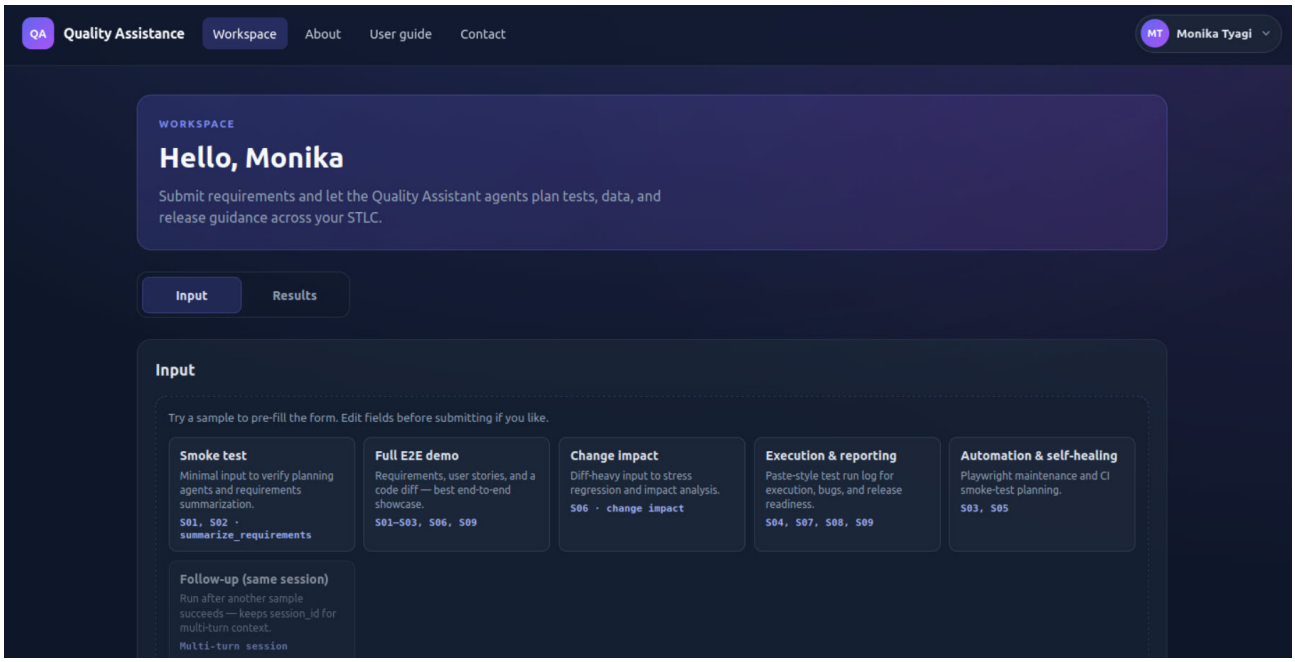


Figure 4.1: Workspace dashboard used to start a quality-assistance request.

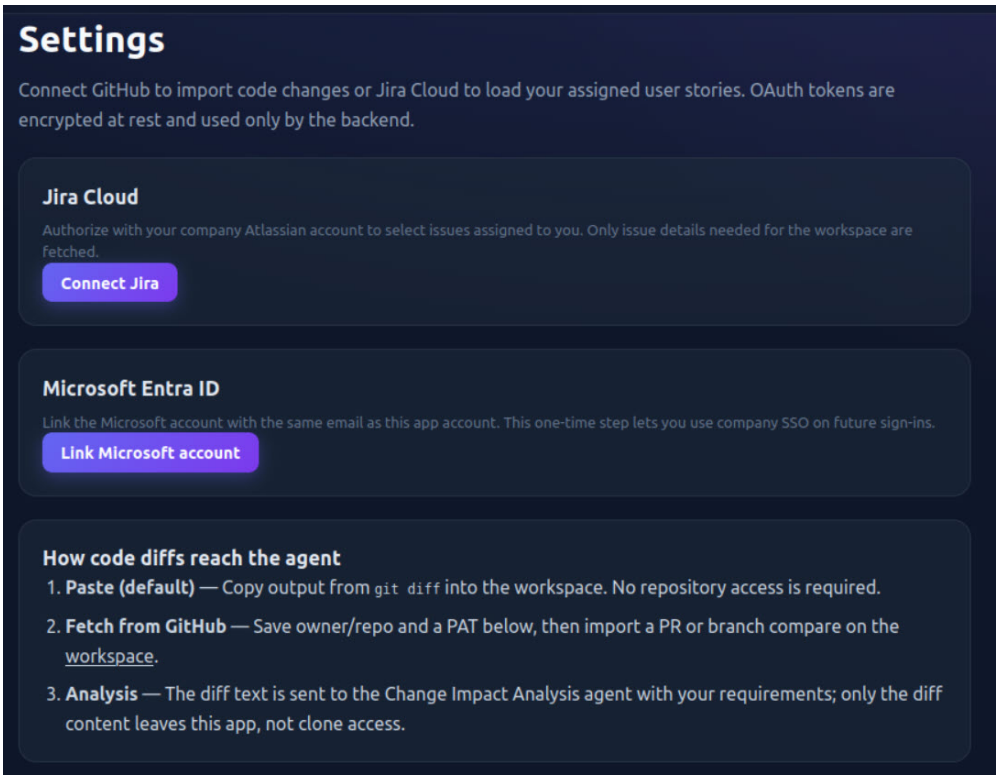


Figure 4.2: Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials.



Figure 4.3: Example request input containing quality context and a user story before agent processing.

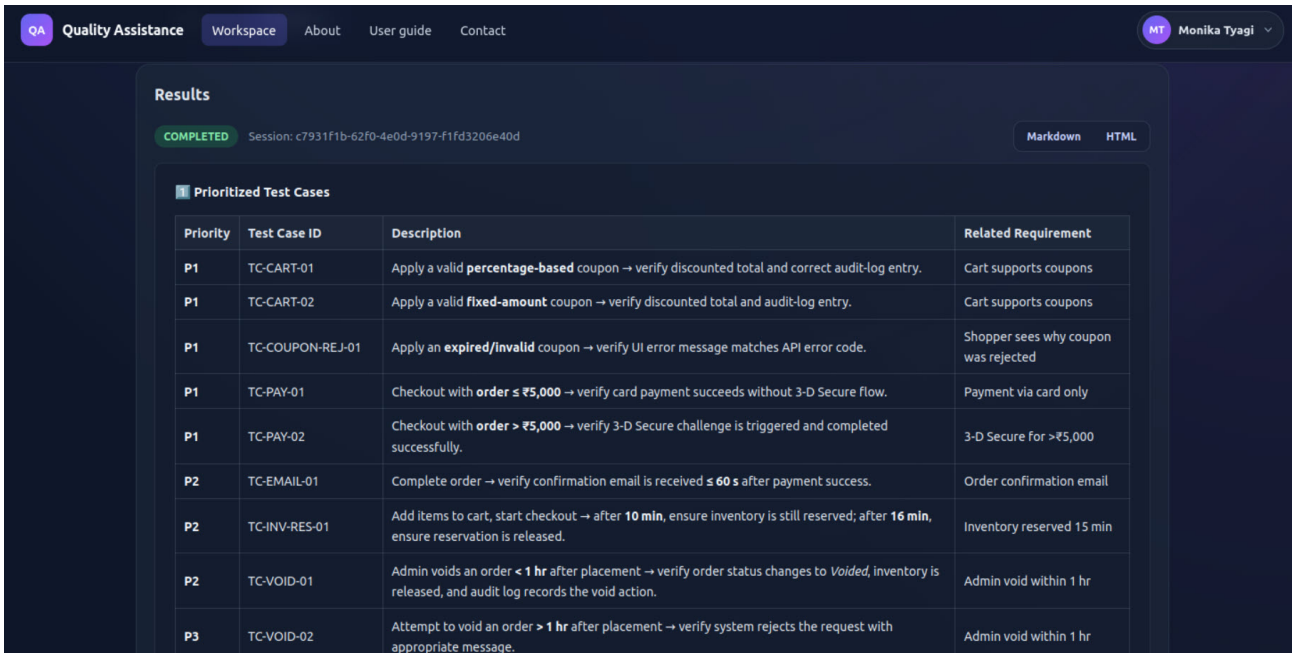


Figure 4.4: Example generated response with prioritized test-case recommendations.



Figure 4.5: Example suggested test data presented as part of the generated response.

3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example automation notes and test-script recommendations.

4 **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example release-risk and readiness analysis based on the submitted context.

4.4 Evaluation Plan

The evaluation should compare three conditions on the same tasks: a manual QA baseline, an existing point-automation workflow where available, and the proposed assistant-supported workflow. The initial phase should use public requirements and open-source repositories or issue trackers, as proposed in the project plan. Any later organizational evaluation should use anonymized artifacts and receive the required data-governance approval.

4.4.1 Dataset and Protocol

For each project, select a bounded set of requirements, user stories, code changes, and available test-run or defect records. Define expected requirement-to-test links and expert-reviewed reference outputs before running the assistant. Use a fixed model, model version, prompt revision, and application configuration for each comparison. Because model outputs can vary, repeat

tasks where practical and retain both raw outputs and review decisions. The reviewer should be blind to the workflow condition when rating generated artifacts, where feasible.

4.4.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.2: Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed.

4.4.3 Analysis and Validity

For quantitative comparisons, report sample sizes, distributions, confidence intervals, and the selected statistical tests. Analyze error categories as well as aggregate scores. Conduct qualitative review of omissions, unsupported statements, ambiguous requirements, and cases where the model's recommendation conflicts with expert judgment. Record the model and prompt versions so that repeated evaluations remain interpretable.

Threats to validity include dataset representativeness, evaluator subjectivity, model nondeterminism, changes in external model services, contamination between task examples and

prompts, and the possibility that generated cases increase review burden. A small public-data evaluation would support feasibility claims but would not alone establish generalization to enterprise workflows.

4.5 Results and Discussion

The implementation evidence supports the claim that the repository contains an integrated application capable of receiving quality context, persisting a request, invoking an orchestrated agent, and returning a response. The specialist registry and sample dashboard scenarios demonstrate breadth across several STLC tasks. Integration and deployment code further establish a path toward connecting the prototype with common engineering systems.

No experiment results, benchmark dataset, measured time savings, coverage percentage, defect-detection rate, or user-study result were provided in the proposal, presentation, or inspected source materials. Accordingly, this report makes no quantitative claim that the system improves quality or reduces effort. The key result is the implemented software artifact and an evaluation protocol that can test those hypotheses. The distinction is important: an agent can generate plausible test cases without proving that they are complete, valid, or effective at finding defects.

4.6 Limitations

- The quality of recommendations depends on the completeness and accuracy of user-supplied context and the selected model.
- Cross-agent routing and synthesis are predominantly prompt-driven; formal inter-stage schemas and independent quality gates are not evidenced.
- Agent session state uses an in-memory runner and is not a durable conversation store across process restarts.
- The system does not autonomously execute external test suites, modify scripts, or enforce release approvals.

- Persistent retrieval over requirements, test assets, and historical defects and a Jenkins or equivalent CI/CD integration are future work.
- The available repository tests address selected settings and service behavior; comprehensive user-interface, model-quality, load, and end-to-end tests remain necessary.
- Production deployment and organization-specific security and privacy approval are not demonstrated by the deployment planning documents.

Chapter 5

Conclusions

This project addresses fragmentation across software quality activities by implementing an AI-assisted application for STLC-oriented analysis. The prototype combines a React and TypeScript frontend, a FastAPI and PostgreSQL backend, and a Google ADK agent service. A root agent can route requests among nine specialists covering test design, test data, automation recommendations, execution analysis, self-healing suggestions, change impact, defect reports, test summaries, and release readiness. The application also supports user authentication, request persistence, Jira Cloud issue context, GitHub diffs, optional Microsoft Entra SSO, and configurable model providers.

The project demonstrates the feasibility of bringing these capabilities into one user workflow. It does not yet demonstrate, through controlled empirical results, that the approach increases coverage, lowers total quality-engineering effort, or detects more defects. The agents provide decision support and should remain subject to human review. In particular, execution analysis is not the same as running a test suite, a script-healing recommendation is not an applied code change, and a generated readiness assessment is not an approval gate.

5.1 Future Work

The next phase should prioritize:

1. introducing structured outputs and explicit validation between specialist stages;

2. adding human approval and edit checkpoints for test cases and higher-impact recommendations;

3. evaluating requirement-to-test traceability, correctness, review effort, latency, and cost against a documented baseline;

4. testing with public projects and, after approval, anonymized organizational artifacts;

5. investigating retrieval over approved requirements, test repositories, and historical defect data;

6. integrating safely with CI/CD test execution and preserving durable, auditable session history; and

7. conducting load, reliability, accessibility, privacy, and security reviews before broader deployment.

The central research question is therefore not whether an LLM can produce a plausible testing artifact, but whether an orchestrated assistant can do so consistently, with useful traceability and acceptable review effort. The implemented prototype provides a basis for answering that question through controlled evaluation.

Appendix

5.A Representative Request and Response Flow

A representative request begins with an authenticated user entering one or more of the following: requirements, user stories, and a code diff. The frontend submits the context to the backend assistance route. The backend associates the request with the current user, saves it with a processing status, and builds an agent message. The agent service resolves a new or existing session, invokes the root agent, and gathers text from the agent events. The backend then stores the response and final status and returns the request and session identifiers to the frontend. A follow-up can send the session identifier to continue the agent interaction.

5.A Reproducibility Checklist for a Future Evaluation

- Record the application revision, model provider, model identifier, prompt revision, and runtime configuration.
- Preserve anonymized input artifacts and the expected reference cases used for scoring.
- Use the same task set and review rubric for each baseline and assistant condition.
- Record generation time separately from human editing and review time.
- Track accepted, edited, and rejected generated items, with reasons.
- Report both aggregate scores and failure cases; do not omit unsafe or misleading outputs.
- Obtain approval before using organizational requirements, source code, or defect records.

5.A Project Materials

The project proposal defines the motivation, questions, phased evaluation plan, and work plan. The mid-term presentation describes the intended agent framework and early project status. The implementation and deployment documentation in the repository provide the source for the as-built architecture and current feature qualifications in this report.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 5

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 5

[3] Monika Tyagi. Ai-driven quality assistant agent for STLC automation: M.tech. (online) project proposal and work plan, 2026. Unpublished project proposal, Indian Institute of Science. 2, 8

[4] Monika Tyagi. Ai-driven quality assistant: Mid-term project presentation, 2026. Unpublished project presentation, Indian Institute of Science. 2, 8

[5] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 6

[6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 6

[7] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan

Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 6

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 6

[9] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 7

[10] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 10

[11] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 11